

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent
Kind Code
Date of Patent
Inventor(s)

12417770
B2
September 16, 2025
Ding; Shaojin et al.

Unified cascaded encoder ASR model for dynamic model sizes

Abstract

An automated speech recognition (ASR) model includes a first encoder, a first encoder, a second encoder, and a second decoder. The first encoder receives, as input, a sequence of acoustic frames, and generates, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames. The first decoder receives, as input, the first higher order feature representation generated by the first encoder, and generates a first probability distribution over possible speech recognition hypotheses. The second encoder receives, as input, the first higher order feature representation generated by the first encoder, and generates a second higher order feature representation for a corresponding first higher order feature frame. The second decoder receives, as input, the second higher order feature representation generated by the second encoder, and generates a second probability distribution over possible speech recognition hypotheses.

Inventors: Ding; Shaojin (Mountain View, CA), He; Yangzhang (Mountain View, CA), Wang; Xin (Mountain View, CA), Wang; Weiran (Palo Alto, CA), Strohman; Trevor (Mountain View, CA), Sainath; Tara N. (Jersey City, NJ), Prabhavalkar; Rohit Prakash (Palo Alto, CA), David; Robert (Mountain View, CA), Panigrahy; Rina (Mountain View, CA), Botros; Rami (Mountain View, CA), Liang; Qiao (Mountain View, CA), McGraw; Ian (Mountain View, CA), Zhao; Ding (Mountain View, CA), Hwang; Dongseong (Mountain View, CA)

Applicant: Google LLC (Mountain View, CA)

Family ID: 1000008819892

Assignee: Google LLC (Mountain View, CA)

Appl. No.: 18/182925

Filed: March 13, 2023

Prior Publication Data

Document Identifier

Publication Date

Related U.S. Application Data

us-provisional-application US 63269703 20220321

Publication Classification

Int. Cl.: G10L15/32 (20130101); G10L15/16 (20060101); G10L15/22 (20060101)

U.S. Cl.:

CPC G10L15/32 (20130101); G10L15/16 (20130101); G10L15/22 (20130101); G10L2015/223 (20130101)

Field of Classification Search

USPC: None

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
11810552	12/2022	Moritz	N/A	G06N 3/0455
2020/0234713	12/2019	Gowda et al.	N/A	N/A
2020/0349922	12/2019	Peyser	N/A	G10L 15/063
2021/0312294	12/2020	Kurata	N/A	G06F 18/24
2022/0122622	12/2021	Narayanan	N/A	G10L 25/30
2022/0208179	12/2021	Kurata	N/A	G10L 15/16
2022/0310062	12/2021	Sainath	N/A	G06N 3/088
2023/0017503	12/2022	Moritz	N/A	G10L 15/30
2023/0109407	12/2022	Hu	704/232	G10L 15/02
2023/0306958	12/2022	Zhang	N/A	G10L 15/16
2023/0326461	12/2022	Ding	704/270.1	G06N 3/044
2024/0169981	12/2023	Huang	N/A	G10L 15/16
2024/0290320	12/2023	Huang	N/A	G10L 15/16

OTHER PUBLICATIONS

Narayanan A, Sainath TN, Pang R, Yu J, Chiu CC, Prabhavalkar R, Varianni E, Strohman T. Cascaded encoders for unifying streaming and non-streaming ASR. arXiv preprint arXiv:2010.14606. Oct. 2, 20207 (Year: 2020). cited by examiner

Gao Z, Zhang S, Lei M, McLoughlin I. Universal asr: Unifying streaming and non-streaming asr using a single encoder-decoder model. arXiv preprint arXiv:2010.14099. Oct. 27, 2020.) (Year: 2020). cited by examiner

Narayanan A, Sainath TN, Pang R, Yu J, Chiu CC, Prabhavalkar R, Varianni E, Strohman T. Cascaded encoders for unifying streaming and non-streaming ASR. arXiv preprint arXiv:2010.14606. Oct. 27, 2020. (Year: 2020). cited by examiner

International Search Report and Written opinion for the related Application No. PCT/US2023/064253, dated May 8, 2023, 64 pages. cited by applicant

Shaojin Ding et al: “A Unified Cascaded Encoder ASR Model for Dynamic Model Sizes”, arxiv.org, Cornell University Library, 201 Olin Library Cornell University Ithaca, NY 14853, Apr. 13, 2022 (Apr. 13, 2022), XP091202853, section '1. Introduction; p. 1, right-hand column section 2. Method; p. 2, left-hand column, paragraph 2—p. 3, left-hand column, paragraph 2, figures 2,3, section '3.2. Implementation details; p. 3, right-hand column. 5 pages. cited by applicant Narayanan Arun et al: “Cascaded Encoders for Unifying Streaming and Non-Streaming ASR”, ICASSP 2021—2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, Jun. 6, 2021 (Jun. 6, 2021), pp. 5629-5633, XP033954311, DOI: 10.1109/ICASSP39728. 2021.9414607, [retrieved on Apr. 22, 2021] section '2. Model Description'; p. 5630, 5 pages. cited by applicant

Primary Examiner: Kim; Jonathan C

Attorney, Agent or Firm: Honigman LLP

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS (1) This U.S. patent application claims priority under 35 U.S.C. § 119(e) to U.S. Provisional Application 63/269,703, filed on Mar. 21, 2022. The disclosure of this prior application is considered part of the disclosure of this application and is hereby incorporated by reference in its entirety.

TECHNICAL FIELD

(1) This disclosure relates to an on-device end-to-end model for simplified streaming and non-streaming automated speech recognition (ASR).

BACKGROUND

(2) Automated speech recognition (ASR) systems have evolved from multiple models where each model had a dedicated purpose to integrated models where a single neural network is used to directly map an audio waveform (i.e., input sequence) to an output sentence (i.e., output sequence). This integration has resulted in a sequence-to-sequence approach, which generates a sequence of words (or graphemes) when given a sequence of audio features. With an integrated structure, all components of a model may be trained jointly as a single end-to-end (E2E) neural network. Here, an E2E model refers to a model whose architecture is constructed entirely of a neural network. A fully neural network functions without external and/or manually designed components (e.g., finite state transducers, a lexicon, or text normalization modules). Additionally, when training E2E models, these models generally do not require bootstrapping from decision trees or time alignments from a separate system. These E2E automatic speech recognition (ASR) systems have made tremendous progress, surpassing conventional ASR systems in several common benchmarks including word error rates (WER). The architecture of E2E ASR models are largely application dependent. For instance, a number of applications that involve user interaction, such as voice-search or on-device dictation, require the model to perform recognition in a streaming fashion. Other applications, like offline video captioning, do not require the model to be streaming and can make use of future context to improve performance. Moreover, the model size for an on-device ASR system is largely dependent on the hardware constraints of the device. For example, vehicle devices equipped with processing hardware with greater computational output may support a larger ASR model size, whereas a mobile device with limited processing hardware may benefit from a smaller ASR model size. Existing E2E approaches typically include separate ASR models for streaming and non-streaming applications. Managing multiple models for different applications provides unique challenges and can lead to slower execution times of the ASR systems.

SUMMARY

(3) One aspect of the present disclosure provides an automatic speech recognition (ASR) model that includes a first encoder, a first decoder, a second encoder, and a second decoder. The first encoder receives, as input, a sequence of acoustic frames, and generates, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames. The first decoder receives, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses. The second encoder receives, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, a second higher order feature representation for a corresponding first higher order feature frame. The second decoder receives, as input, the second higher order feature representation generated by the second encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses.

(4) Implementations of the disclosure may include one or more of the following optional features. In some implementations, the first decoder generates partial speech recognition results based on the first probability distribution over possible speech recognition hypotheses. In some examples, the first encoder includes a causal encoder including one of a plurality of unidirectional long short-term memory (LSTM) layers, a plurality of conformer layers, or a plurality of transformer layers. Additionally or alternatively, the second encoder includes a non-causal encoder including one of a plurality of unidirectional long short-term memory (LSTM) layers, a plurality of conformer layers, or a plurality of transformer layers.

(5) In some examples, the first decoder includes a prediction network and a joint network. The prediction receives, as input, a sequence of non-blank symbols output by a final softmax layer, and generates, at each of the plurality of output steps, a dense representation. The joint network receives, as input, the dense representation generated by the prediction network at each of the plurality of output steps and the first higher order feature representation generated by the first encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, the first probability distribution over possible speech recognition hypotheses. In these examples, the prediction network includes a long short-term memory (LSTM)-based prediction network, or a V2 embedding look-up table.

(6) In some implementations, the second decoder includes a prediction network and a joint network. The prediction network receives, as input, a sequence of non-blank symbols output by a final softmax layer, and generates, at each of the plurality of output steps, a dense representation. The joint network receives, as input, the dense representation generated by the prediction network at each of the plurality of output steps and the second higher order feature representation generated by the second encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, the second probability distribution over possible speech recognition hypotheses. In these implementations, the prediction network includes a long short-term memory (LSTM)-based prediction network, or a V2 embedding look-up table.

(7) In some examples, the first encoder includes a greater number of parameters than the second encoder. In some implementations, the ASR model further includes a third encoder and a third decoder. The third encoder receives, as input, the second higher order feature representation generated by the second encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, a third higher order feature representation for a corresponding second higher order feature representation. The third decoder receives, as input, the third higher order feature representation generated by the third encoder at each of the plurality of output steps, and generates, at each of the plurality of output steps, a third probability distribution over possible speech recognition hypotheses.

(8) Another aspect of the present disclosure provides a computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations that include receiving, as input to an automatic speech recognition (ASR) model, a sequence of acoustic frames, and performing, using the ASR model, streaming speech recognition and non-streaming speech recognition on the sequence of acoustic frames. The operations also include generating, by a first encoder, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames, and receiving, as input to both a first decoder and a second encoder, the first higher order feature representation generated by the first encoder at each of the plurality of output steps. The operations also include generating, by the second encoder, at each of the plurality of output steps, a second higher order feature representation for a corresponding first higher order feature representation, and generating, by the first decoder, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses. The operations further include receiving, as input to a second decoder, the second higher order feature representation generated by the second encoder at each of the plurality of output steps, and generating, by the second decoder, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses.

(9) This aspect may include one or more of the following optional features. In some implementations, the operations further include generating partial speech recognition results based on the first probability distribution over possible speech recognition hypotheses. In some examples, the first encoder includes a causal encoder including one of a plurality of unidirectional long short-term memory (LSTM) layers, a plurality of conformer layers, or a plurality of transformer layers. Additionally or alternatively, the second encoder includes a non-causal encoder including one of a plurality of unidirectional long short-term memory (LSTM) layers, a plurality of conformer layers, or a plurality of transformer layers.

(10) In some implementations, when the ASR model is operating in a streaming mode, the operations further include receiving, as input to a prediction network of the first decoder, a sequence of non-blank symbols output by a final softmax layer, generating, by the prediction network of the first decoder, at each of the plurality of output steps, a dense representation. Here, the operations further include receiving, as input to a joint network of the first decoder, the dense representation generated by the prediction network at each of the plurality of steps and the first higher order feature representation generated by the first encoder at each of the plurality of output steps, and generating, by the joint network of the first decoder, at each of the plurality of output steps, the first probability distribution over possible speech recognition hypotheses. In these implementations, the prediction network of the first decoder includes a long short-term memory (LSTM)-based prediction network, or a V2 embedding look-up table.

(11) In some examples, when the ASR model is operating in a non-streaming mode, the operations also include receiving, as input to a prediction network of the second decoder, a sequence of non-blank symbols output by a final softmax layer, and generating, by the prediction network of the second decoder, at each of the plurality of output steps, a dense representation. Here, the operations further include receiving, as input to a joint network of the second decoder, the dense representation generated by the prediction network at each of the plurality of output steps and the second higher order feature representation generated by the second encoder at each of the plurality of output steps, and generating, by the joint network of the second decoder, at each of the plurality of output steps, the second probability distribution over possible speech recognition hypotheses. In these examples, the prediction network of the second decoder includes a long short-term memory (LSTM)-based prediction network, or a V2 embedding look-up table.

(12) In some implementations, the first encoder includes a greater number of parameters than the second encoder. In some examples, the operations also include, while performing the streaming speech recognition and the non-streaming speech recognition on the sequence of acoustic frames,

receiving, as input to a third encoder, the second higher order feature representation generated by the second encoder at each of the plurality of output steps. In these implementations, the operations further include generating, by the third encoder, at each of the plurality of output steps, a third higher order feature representation for a corresponding second higher order feature representation, receiving, as input to a third decoder, the third higher order feature representation generated by the third encoder at each of the plurality of output steps, and generating, by the third decoder, at each of the plurality of output steps, a third probability distribution over possible speech recognition hypotheses.

(13) The details of one or more implementations of the disclosure are set forth in the accompanying drawings and the description below. Other aspects, features, and advantages will be apparent from the description and drawings, and from the claims.

Description

DESCRIPTION OF DRAWINGS

(1) FIGS. 1A and 1B are schematic views of a system using a cascaded encoders model architecture for automatic speech recognition.

(2) FIGS. 2A-2D are schematic views of the cascaded encoders model architecture.

(3) FIGS. 3A and 3B are schematic views of an example training process for promoting a cascaded encoders model to learn consistent predictions on both streaming automatic speech recognition and non-streaming automatic speech recognition.

(4) FIG. 4 is a flowchart of an example arrangement of operations for a method of implementing the cascaded encoders model for both streaming automatic speech recognition and non-streaming automatic speech recognition.

(5) FIG. 5 is a schematic view of an example computing device that may be used to implement the systems and methods described herein.

(6) Like reference symbols in the various drawings indicate like elements.

DETAILED DESCRIPTION

(7) End-to-end (E2E) automatic speech recognition (ASR) models are traditionally structured to operate in either a streaming mode or a non-streaming mode. Conventionally, an E2E ASR model includes an encoder and a decoder as the main components. Applications that involve end-user interaction, like voice-search or on-device dictation, may require the model to perform recognition in a streaming fashion, where the words are expected to be output as they are spoken with as little latency as possible. This prevents the use of models that use future context to improve accuracy, such as bi-directional LSTMs. By contrast, applications such as offline video captioning do not require streaming recognition and may make full use of any available future context to improve performance. Moreover, the model size for an on-device ASR system is largely dependent on the hardware constraints of the device. For example, vehicle devices equipped with processing hardware with greater computational output may support a larger ASR model size, whereas a mobile device with limited processing hardware may benefit from a smaller ASR model size. Even on the same device, a large ASR model size may be used for short-form applications such as voice search, whereas a medium ASR model size or small ASR model size may be better suited for long-running applications such as dictation or video captioning to maintain low power consumption.

(8) Implementations herein are directed toward a single E2E ASR model that uses cascaded encoders and two separate decoders to enable operation in both streaming and non-streaming modes with varying model parameters. The cascaded encoders include a streaming encoder and a non-streaming encoder, and the two separate decoders include a streaming decoder and a non-streaming decoder that are configured to learn to decode the output from the respective streaming encoder or the output from the respective non-streaming encoder. In addition to ASR models, the

architecture can apply to other models such as machine translation that implement both streaming and non-streaming modes.

(9) Referring to FIGS. 1A and 1B in some implementations, a system **100**, **100a-b** includes a user **104** interacting with speech-enabled user device **10** (also referred to as a device **10** or a user device **10**) within a speech environment. In the system **100**, the user's **104** manner of interacting with the user device **10** may be through voice input. The user device **10** is configured to capture sounds (e.g., streaming audio data) from one or more users **104** within the speech environment. Here, the streaming audio data may refer to a spoken utterance **106** by the user **104** that functions as an audible query, a command for the device **10**, or an audible communication captured by the device **10**. Speech-enabled systems of the device **10** may field the query or the command by answering the query and/or causing the command to be performed/fulfilled by one or more downstream applications.

(10) The user device **10** may correspond to any computing device associated with a user **104** and capable of receiving audio data. Some examples of user devices **10** include, but are not limited to, mobile devices (e.g., mobile phones, tablets, laptops, etc.), computers, wearable devices (e.g., smart watches), smart appliances, internet of things (IoT) devices, vehicle infotainment systems, smart displays, smart speakers, etc. The user device **10** includes data processing hardware **12** and memory hardware **14** in communication with the data processing hardware **12** and stores instructions, that when executed by the data processing hardware **12**, cause the data processing hardware **12** to perform one or more operations. The user device **10** further includes an audio system **16** with an audio capture device (e.g., microphone) **16**, **16a** for capturing and converting spoken utterances **106** within the speech environment into electrical signals and a speech output device (e.g., a speaker) **16**, **16b** for communicating an audible audio signal (e.g., as output audio data from the device **10**). While the user device **10** implements a single audio capture device **16a** in the example shown, the user device **10** may implement an array of audio capture devices **16a** without departing from the scope of the present disclosure, whereby one or more capture devices **16a** in the array may not physically reside on the user device **10**, but be in communication with the audio system **16**.

(11) In the system **100**, an automated speech recognition (ASR) system **109** implementing an ASR model **200** (also referred to as the model **200**) resides on the user device **10** of the user **104** and/or on a remote computing device **60** (e.g., one or more remote servers of a distributed system executing in a cloud-computing environment) in communication with the user device **10** via a network **40**. The remote computing device **60** may include remote data processing hardware **62** (e.g., remote servers or CPUs) and/or remote memory hardware **64** (e.g., remote databases or other storage hardware). The user device **10** and/or the remote computing device **60** also includes an audio subsystem **108** configured to receive the utterance **106** spoken by the user **104** and captured by the audio capture device **16a**, and to convert the utterance **106** into a corresponding digital format associated with input acoustic frames **110** capable of being processed by the ASR system **109**. In the example shown in FIG. 1A, the user **104** speaks a respective utterance **106** and the audio subsystem **108** converts the utterance **106** into corresponding audio data (e.g., acoustic frames) **110** for input to the ASR system **109**. Thereafter, the model **200** receives, as input, the audio data **110** corresponding to the utterance **106**, and generates/predicts, as output, a corresponding transcription **120** (also referred to as a recognition result/hypothesis **120**) of the utterance **106**. As described in greater detail below (e.g., FIGS. 3A and 2B), the model **200** may be trained in two-stage training to simplify the process of training the model **200** to operate in a streaming and a non-streaming mode. The model **200** also includes a first decoder **204** (also referred to as a streaming decoder **204**) and a second decoder **206** (also referred to as a non-streaming decoder **206**) which enable the model **200** to operate in a streaming mode and a non-streaming mode (e.g., in contrast with two separate models where each model is dedicated to either a streaming mode or non-streaming mode).

(12) For instance, as shown in FIG. 1A, a digital assistant application **50** executing on the user device **10** may require the speech recognition to be streaming such that words, word pieces, and/or individual characters appear on the screen as soon as they are spoken. Additionally, it is also likely that the user **104** of the user device **10** has a low tolerance for latency when issuing queries for the digital assistant application **50** to perform. In these scenarios where the application demands minimal latency and benefits from a small model size, the model **200** operates in a streaming mode where the model **200** may provide streaming transcription capabilities in real-time as the user **104** is speaking the utterance **106**. On the other hand, when the user **104** has a higher tolerance for speech recognition latency and/or the utterance **106** to be recognized is associated with long-form speech (i.e., referring to speech consisting of full paragraphs or multiple sentences) and the capacity for a larger model, the same model **200** may operate in a non-streaming mode and may leverage a prediction network to provide an accurate transcription **120**, but incur increased latency. Moreover, in scenarios where a user **104** has an even higher tolerance for speech recognition latency and/or the accuracy is of the utmost importance, the model **200** may operate in a non-streaming mode that uses additional context to improve accuracy at the cost of increased speech recognition latency. Accordingly, the ASR system **109** may implement the ASR model **200** for a multitude of different speech recognition tasks to provide both streaming and non-streaming transcription capabilities without having to leverage separately trained ASR models on a task-by-task basis. Furthermore, using the single ASR model **200** for varying speech recognition tasks reduces the computational requirements to operate separate ASR models.

(13) In some implementations, the model **200** performs streaming speech recognition on the audio data **110** first and then performs non-streaming speech recognition on the output of the streaming encoder. For instance, in the example shown, the model **200** performs streaming speech recognition on the audio data **110** using a first encoder (i.e., a low latency encoder (FIG. 2B)) to produce partial speech recognition results **120**, **120a**, and non-streaming speech recognition on the encoded audio data **110** using a second encoder (i.e., a high latency encoder (FIG. 2C)) to produce a final speech recognition result **120**, **120b**. Notably, the model **200** outputs the partial speech recognition results **120a** as the audio data **110** is received and subsequently outputs the final speech recognition result **120b** once all the audio data **110** is received after the user **104** finishes speaking the utterance **106**. Thus, output of the final speech recognition result **120b** for the input utterance **106** may be delayed from the partial speech recognition results **120a**.

(14) The user device **10** and/or the remote computing device **60** also executes a user interface generator **107** configured to present a representation of the transcription **120** of the utterance **106** to the user **104** of the user device **10**. As described in greater detail below, the user interface generator **107** may display the partial speech recognition results **120a** in a streaming fashion during time **1** and subsequently display the final speech recognition result **120b** during time **2**. In some configurations, the transcription **120** output from the ASR system **109** is processed, e.g., by a natural language understanding (NLU) module executing on the user device **10** or the remote computing device **60**, to execute a user command/query specified by the utterance **106**. Additionally or alternatively, a text-to-speech system (not shown) (e.g., executing on any combination of the user device **10** or the remote computing device **60**) may convert the transcription **120** into synthesized speech for audible output by the user device **10** and/or another device.

(15) In the example system **100a** of FIG. 1A, the user **104** in the speech environment interacts with a program or application **50** (e.g., the digital assistant application **50a**) of the user device **10** that uses the ASR system **109**. For instance, FIG. 1A depicts the user **104** communicating with the digital assistant application **50a** and the digital assistant application **50a** displaying a digital assistant interface **18** on a screen of the user device **10** to depict a conversation between the user **104** and a digital assistant of the digital assistant application **50a**. In this example, the user **104** asks the digital assistant application **50a**, "What song is playing right now?" This question from the user

104 is a spoken utterance **106** captured by the audio capture device **16a** and processed by audio systems **16** of the user device **10**. In this example, the audio system **16** receives the spoken utterance **106** and converts it into acoustic frames **110** for input to the ASR system **109**.

(16) Continuing with the example, the model **200**, while receiving the acoustic frames **110** corresponding to the utterance **106** as the user **104** speaks, encodes the acoustic frames **110** using a first encoder **210** (i.e., FIGS. 2A and 2B) and then decodes an encoded representation of the acoustic frames **110** using a first decoder **204** (FIGS. 2A and 2B) into the partial speech recognition results **120a**. During time **1**, the user interface generator **107** presents, via the digital assistant interface **18**, a representation of the partial speech recognition results **120a** of the utterance **106** to the user **104** of the user device **10** in a streaming fashion such that words, word pieces, and/or individual characters appear on the screen as soon as they are spoken.

(17) After all (or some amount) of the acoustic frames **110** corresponding to the utterance **106** are received, and the first encoder **210** has encoded these acoustic frames **110**, the second encoder **220** (i.e., FIGS. 2A and 2C) encodes the encoding output from the first encoder **210** to generate an encoding for the set of acoustic frames **110** corresponding to the utterance **106** already encoded by the first encoder **210**. A second decoder **206** (i.e., FIGS. 2A and 2C) then decodes the acoustic frames **110** that have been encoded by the second encoder **220** into a final speech recognition result **120b**. For example, when the first encoder **210** encodes all of the acoustic frames **110** corresponding to the utterance **106** (e.g., as the acoustic frames **110** are received), the second encoder **220** encodes all of the acoustic frames **110** that have been encoded by the first encoder **210**. In this respect, by encoding over multiple encoded acoustic frames **110**, the second encoder **220** is able to provide greater contextual awareness (e.g., by receiving representations of all of the acoustic frames **110** for the utterance **106**) in a non-streaming fashion which may potentially reconcile or correct aspect(s) of the utterance **106** missed or misinterpreted by the streaming nature of the first encoder **210**. In some examples, an indication, such as an endpoint, that identifies that the user **104** has finished speaking the utterance **106** triggers the second encoder **220** of the model **200** to encode all the acoustic frames **110**.

(18) During time **2**, the user interface generator **107** presents, via the digital assistant interface **18**, a representation of the final speech recognition result **120b** of the utterance **106** to the user **104** of the user device **10**. In some implementations, the user interface generator **107** replaces (or modifies) the representation of the partial speech recognition results **120a** with the representation of the final speech recognition result **120b**. For instance, the final speech recognition result **120b** is presumed to be more accurate than the partial speech recognition results **120a** and the final speech recognition result **120b** is ultimately displayed as the transcription **120** in order to fix any terms that may have been misrecognized in the partial speech recognition results **120a**. In this example, the streaming partial speech recognition results **120a** output by the model **200** (i.e., the first encoder **210** and the first decoder **204**) and displayed on the screen of the user device **10** at time **1** are associated with low latency and provide responsiveness to the user **104** that his/her query is being processed. Accordingly, the partial speech recognition results **120a** may incorrectly predict that the utterance **106** of the user **104** is “What song is play right now?” The final speech recognition result **120b** output by the model **200** (i.e., the cascaded encoders **202** and the second decoder **206**) and displayed on the screen at time **2** at increased latency improves the speech recognition quality in terms of accuracy by identifying that the user **104** said “playing.” However, since the user interface generator **107** displays the partial speech recognition results as the user speaks the utterance **106**, the higher latency associated with producing, and ultimately displaying the final recognition result **120b** is less noticeable to the user **104**.

(19) In the example shown in FIG. 1A, the digital assistant application **50a** may respond to the question posed by the user **104** using natural language processing. Natural language processing generally refers to a process of interpreting written language (e.g., the partial speech recognition results **120a** and/or the final speech recognition result **120b**) and determining whether the written

language prompts any action. In this example, the digital assistant application **50a** uses natural language processing to recognize that the question from the user **104** regards the user's environment and more particularly a song playing in the user's vicinity. By recognizing these details with natural language processing, the automated assistant returns a response **19** to the user's query where the response **19** states, "Cloudplay is playing right now." In some configurations, natural language processing occurs on the remote computing device **60** in communication with the data processing hardware **12** of the user device **10**.

(20) FIG. 1B is another example of speech recognition with the ASR system **109** of the speech environment **100b**. As shown in the example, the user **104** interacts with a voicemail application **50**, **50b** displaying a voicemail application interface **18**, **18b** on the screen of the user device **10** to transcribe a voicemail that was left for the user **104** by Jane Doe. In this example, latency is not important. Without a concern for latency, the model **200** of the ASR system **109** is able to take advantage of the full context of the audio by waiting until all of the acoustic frames **110** corresponding to the voicemail are generated. This voicemail scenario also illustrates how the model **200** is capable of handling a long-form of speech because a voicemail is often multiple sentences or even several paragraphs. The ability to handle long-form speech is particularly advantageous over other ASR models, such as multi-pass models with LAS decoders, because these models often incur reduced performance (e.g., a higher word deletion rate on long-form speech) when recognizing long-form speech. For instance, by using a dedicated second decoder **206** trained on the features of the second encoder **220** in combination with cascaded encoders **202** (e.g., the first encoder **210** and the second encoder **220**), the model **200** is able to consume the full context of the input audio when performing speech recognition on long-form speech without incurring a reduction performance. As discussed in further detail below, the cascading encoders **202** of the model **200** may optionally include a third encoder **230** in cascade with the first encoder **210** and the second encoder **220**, and a dedicated third decoder **208** that decodes the acoustic frames **110** that have been encoded by the third encoder **230** into the final speech recognition result **120b**.

(21) With continued reference to FIG. 1B, as discussed with respect to FIG. 1A, the model **200** encodes the acoustic frames **110** using the first encoder **210** while receiving the acoustic frames **110**. After the model **200** receives all of the acoustic frames **110** and encodes them with the first encoder **210**, the model **200** provides the first encoder output as input to the second encoder **220**. The second encoder **220** encodes the first encoder output before the second decoder **206** generates the final speech recognition result **120b**. Thereafter, the user interface generator **107** presents, via the digital assistant interface **18b**, a representation of the final speech recognition result **120b** without first displaying the partial speech recognition results **120a**. For example, the final speech recognition result **120b** is a transcript of the long-form voicemail from Jane Doe that states "Give me a call back when you get this. Just trying to figure out plans for the New Year."

(22) FIGS. 2A-2D include example models **200a-d** operating in various combinations of streaming and non-streaming modes. Specifically, each of the models **200a-d** include a cascading encoder **202**, a first decoder **204**, and a second decoder **206**. The cascaded encoder **202** refers to a model structure where the encoding pathway includes the encoders **210**, **220** that cascade such that the output of one encoder **210** feeds the input of a second encoder **220** prior to decoding. Here, the encoders **210**, **220** can be cascaded irrespective of the underlying architecture for each encoder. As shown in FIG. 2D, and discussed in more detail below, the model **200** can be generalized to a three-pass model **200** (i.e., a large model) where the cascaded encoder **202** includes a third encoder **230** in cascade with the encoders **210**, **220** and connected to a dedicated third decoder **208**. Here, the output of the second encoder **220** feeds the input of the third encoder **230** prior to decoding by the third decoder **208**.

(23) In some examples, the encoders **210**, **220**, **230** include a stack of 512-dimension conformer layers. In other examples, the first encoder **210** (i.e., a first pass) includes a stack (e.g., six layers) of 256-dimension causal conformer layers, the second encoder **220** (i.e., a second pass) includes a

stack (e.g., six layers) of 512-dimension non-causal conformer layers, and the third encoder **230** (i.e., the third pass) includes a stack (e.g., six layers) of 640-dimension non-causal conformer layers. Causal convolution and left-context attention layers may be used for each conformer layer to strictly restrict the model to use no future inputs. A multi-headed (e.g., 8 heads) attention mechanism may be used in a self-attention layer.

(24) The encoder **210**, when the model **200** operates as a first pass model, includes six conformer layers with around 20 million parameters. The cascaded encoders **210**, **220**, when the model **200** operates as the second pass model, may include 12 conformer layers with around 50 million parameters. Here, the first encoder **210** may include six conformer layers with around 20 million parameters, where the first three layers have no self-attention (e.g., a 92 frame left context and no right context) to prevent the model **200** from using future inputs, while the second encoder **220** may include six conformer layers that take in additional right context (e.g., 5.04 seconds) with around 30 million parameters. The cascaded encoders **210**, **220**, **230**, when the model **200** operates as a third pass model, includes 18 layers with around 110 million parameters, where the third encoder **230** includes six non-causal conformer layers with an additional right context (e.g., 30 frame right context of 900 milliseconds (ms) with around 60 million parameters. Optionally, other types of layers incorporating self-attention mechanisms, such as transformer layers, may be used in lieu of conformer layers. The first encoder **210** may be referred to as a causal encoder and the second encoder **220** and third encoder **230** may each be referred to as a non-causal encoder.

(25) In other implementations, one encoder is constructed with an LSTM structure while the other encoder is constructed using bi-directional LSTM layers or conformer layers (e.g., a conformer-transducer). In other words, the encoders **210**, **220**, **230** may have different architectures or similar architectures. For instance, the cascading encoder **202** may be roughly analogous to an acoustic model (AM) in a traditional ASR system, and may include a recurrent network of stacked Long Short-Term Memory (LSTM) layers. Here, the first encoder **210** is a streaming encoder that includes unidirectional Long Short Term Memory (LSTM) layers while the second encoder **220** is a non-streaming encoder that includes bidirectional LSTM layers or conformer layers. In a cascading encoder **202**, where the encoders **210**, **220**, **230** include LSTM layers, the second encoder **220** that receives the output of the first encoder **210** may take advantage of the LSTM layers of the first encoder **210** such that the second encoder **220** includes fewer LSTM layers than the first encoder **210** (and fewer LSTM layers than a fully non-streaming model). By having fewer LSTM layers, the cascaded encoder **202** may reduce the number of more computationally expensive bidirectional layers making the model **200** more streamlined than simply combining a traditional streaming model with a traditional non-streaming model.

(26) Referring to FIG. 2A, the first encoder **210** reads a sequence of d-dimensional feature vectors (e.g., acoustic frames **110** shown in FIGS. 1A and 1B) $x=(x_{\text{sub.1}}, x_{\text{sub.2}}, \dots, x_{\text{sub.T}})$, where $x_{\text{sub.1}} \in \mathbb{R}^{\text{sup.d}}$, and produces, at each output step, a first higher-order feature representation. This first higher-order feature representation is denoted as $e_{\text{sup.s}}$. Similarly, the second encoder **220** is connected in cascade to the first encoder **210**, and is trained to receive the first higher order feature $e_{\text{sup.s}}$ as input, and output a second higher order feature representation. This second higher order feature representation is denoted as $e_{\text{sup.a}}$. Optionally, the third encoder **230** is connected in cascade to the second encoder **220**, and is trained to receive the second higher order feature representation $e_{\text{sup.a}}$ as input, and output a third higher order feature representation. This third higher order feature representation is denoted as $e_{\text{sup.r}}$. Each of the first encoder **210**, the second encoder **220**, and the third encoder **230** are directly connected to their own dedicated, smaller decoders **204**, **206**, **208**. Specifically, the first encoder **210** is connected to the first decoder **204**, the second encoder **220** is connected to the second decoder **206**, and the third encoder **230** is connected to the third decoder **208**. Accordingly, the first decoder **204** receives the first higher order feature representation $e_{\text{sup.s}}$ input, the second decoder **206** receives the second higher order feature representation $e_{\text{sup.a}}$ as input, and the third decoder **208** receives the third higher order feature

representation e.sup.r as input.

(27) The first decoder **204**, the second decoder **206**, and the third decoder **208** may each include a recurrent neural network-transducer (RNN-T) architecture having a respective 4.4 million parameters based on a 384-dimensional fully-connected joint layer **240**, **240a-c** and a respective 320-dimensional embedding prediction network **250**, **250a-c**. When the model **200** operates in the streaming mode (i.e., FIG. 2B), the joint layer **240a** of the first decoder **204** receives an embedding output from the prediction network **250a** (for the previous prediction y.sub.r-1) and combines it with the first higher order feature representation e.sup.s output from the first encoder **210** in order to produce the first decoder **204** output. When the model **200** operates in the two-pass non-streaming mode (FIG. 2C), the second decoder **206** uses the joint layer **240b** to combine the second higher order feature representations e.sup.a output by the cascaded encoder **202**, as well as an embedding output from the prediction network **250b** (for the previous prediction y.sub.r-1), in order to produce the second decoder **206** output. Similarly, when the model **200** operates in the third-pass non-streaming mode (FIG. 2D), the third decoder **208** uses the joint layer **240c** to combine the third higher order feature representation e.sup.r output by the cascaded encoder **202**, as well as an embedding output from the prediction network **250c** (for the previous prediction y.sub.r-1), in order to produce the third decoder **208** output.

(28) The respective outputs of the decoders **204**, **206**, **208** can be a probability distribution, $P(y_{\text{sub}.i}|y_{\text{sub}.i-1}, \dots, y_{\text{sub}.0}, x)$, over the current sub-word unit, y.sub.i, given the sequence of the N previous non-blank symbols previous units, $\{y_{\text{sub}.i-1}, \dots, y_{\text{sub}.i-N}\}$, and input, x. Although not illustrated, the model **200** may include a Softmax layer that receives the output of the decoders **204**, **206**, **208**. In some implementations, the Softmax layer is separate from the decoders **204**, **206**, **208** and processes the outputs, y.sub.r, from the decoders **204**, **206**, **208**. The output of the Softmax layer is then used in a beam search process to select orthographic elements. In some implementations, a Softmax layer is integrated within each of the decoders **204**, **206**, **208**, such that the output y.sub.r of the respective decoders **204**, **206**, **208** represents the output of the Softmax layer.

(29) The decoders **204**, **206**, **208** are configured to generate, at each output step, a probability distribution over possible speech recognition hypotheses. Stated differently, for each of the decoders **204**, **206**, **208**, the respective joint layer **240** generates, at each output step (e.g., time step), a probability distribution over possible speech recognition hypotheses. Here, the “possible speech recognition hypotheses” correspond to a set of output labels/symbols (also referred to as “speech units”) each representing a grapheme (e.g., symbol/character) or a word piece in a specified natural language. For example, when the natural language is English, the set of output labels may include twenty-seven (27) symbols, e.g., one label for each of the 26-letters in the English alphabet and one label designating a space. Accordingly, each respective joint layer **240** may output a set of values indicative of the likelihood of occurrence of each of a predetermined set of output labels. This set of values can be a vector (e.g., a one-hot vector) and can indicate a probability distribution over the set of output labels. In some cases, the output labels are graphemes (e.g., individual characters, and potentially punctuation and other symbols), but the set of output labels is not so limited. For example, the set of output labels can include wordpieces and/or entire words, in addition to or instead of graphemes. The output labels could also be other types of speech units, such as phonemes or sub-phonemes. The output distribution of the joint layer **240** can include a posterior probability value for each of the different output labels. Thus, if there are 100 different output labels representing different graphemes or other symbols, the output of the joint layer **240** can include 100 different probability values, one for each output label. The probability distribution can then be used to select and assign scores to candidate orthographic elements (e.g., graphemes, wordpieces, and/or words) in a beam search process (e.g., by the Softmax layer) for determining the transcription **120**.

(30) Within each of the decoders **204**, **206**, **208**, the respective prediction network **250** may have

two 2,048-dimensional LSTM layers, each of which is also followed by 640-dimensional projection layer, such that the LSTM-based prediction network may have about 23.4 million parameters. In other words, each prediction network **250** may include LSTM layers. In other configurations, the prediction network **250** may instead include conformer or transformer layers in lieu of LSTM layers. In yet other configurations, the prediction network **250** includes a V2 embedding look up table that includes an embedding prediction network. At each time step, the V2 embedding lookup table may receive, as input, the previous two predictions (e.g., 1-hot vectors) output by the prediction network **250**, compute a respective embedding d.sub.1, d.sub.2 for each of the previous two predictions, and provide a concatenated output [d.sub.1, d.sub.2] to the joint layer **240**. Comparatively, the V2 embedding lookup table may have only about two (2) million parameters, whereas an LSTM-based prediction network may include about 23.4 million parameters. Finally, the prediction network **250** may also be a one-layer neural network with **640** hidden units. The Softmax layer may be composed of a unified word piece or grapheme set that is generated using all unique word pieces or graphemes in a plurality of training utterances **132**, **132a-n** (FIGS. 3A-3B). In some implementations, in order to limit the amount of future context that the ASR model **200** sees, the second encoder **220** uses some number of conformer layers (e.g., two layers) with a particular amount of right context (e.g., five seconds of right context), while the first encoder **210** continues to use LSTM layers. For these implementations, each conformer layer in the second encoder **220** may have 640 units to match the LSTM layers and adds around 10 million additional parameters.

(31) In some implementations, where the model **200** operates in the streaming mode (i.e., the one-pass model), funnel pooling is used to down-sample the input acoustic frames **110**, thereby reducing the number of parameters in the model **200** while preserving performance of the model **200**. For example, a self-attention layer of the first encoder **210** receives, as input, a feature map $x \in \mathbb{R}^{\text{sup.}T \times D}$, where T is the original sequence length and D is the feature dimension. Thereafter, a down-sampled sequence, $x' \in \mathbb{R}^{\text{sup.}T' \times D}$ is generated by average pooling, which is expressed as follows:

$$x' = \text{AvgPool}(x) \quad (1)$$

Where $T' = T/2$. Rather than providing x' as input to the self-attention layer, it is used as query vector q , where the key vector k and the value vector v are based on the input feature map x which maps to $y \in \mathbb{R}^{\text{sup.}T' \times D}$ as the output features of the self-attention layer as follows:

$$y = \text{SelfAttention}(q=x', kv=x) \quad (2).$$

(32) Continuing with the example in FIG. 2A, in some implementations, the model **200a** operates in both the streaming and non-streaming modes in parallel. When operating in both streaming and non-streaming mode at the same time, the model **200a** first performs streaming speech recognition on the audio data **110** using the first encoder **210** to generate the first higher order representation e.sup.s for both the second encoder **220** and the first decoder **204**. The first decoder **204** then produces the partial speech recognition results **120**, **120a** based on the first higher order representation e.sup.s. The model **200b** also performs non-streaming speech recognition on the encoded audio data **110** where the second encoder **220** uses the first higher order representation e.sup.s received from the first encoder **210** to generate the second higher order representation e.sup.a. The second decoder **206** then produces the final speech recognition result **120**, **120b** based on the second higher order representation e.sup.a. As noted by the time, the first decoder **204** outputs the partial speech recognition results **120a** in a streaming fashion using the output from the first encoder **210**, and then the model **200a** waits until the utterance is complete before operating in the non-streaming mode where the second decoder **206** uses the output from the second encoder **220** to produce the final speech recognition result **120b**. Thus, the final speech recognition result **120b** for the input utterance **106** may be delayed from the partial speech recognition results **120a**.

(33) Referring to FIG. 2B, in some implementations, the model **200b** operates only in the streaming mode (i.e., a single-pass mode). This may occur, for instance, when the user **104** is using

applications such as voice-search or on-device dictation, which require as little latency as possible. Here, the model **200b** performs streaming speech recognition on the audio data **110** using only the first encoder **210** to generate the first higher order feature representation e.sup.s for the first decoder **204**. The first decoder **204** then produces the partial speech recognition results **120**, **120a**. Because the streaming mode of the model **200b** produces the partial speech recognition results **120**, **120a** quickly, the inaccuracy of the term “play” is generally acceptable to users **104**.

(34) Referring to FIG. 2C, in some implementations the model **200c** operates only in the non-streaming mode (i.e., a two-pass model). The non-streaming mode may occur, for instance, in non-latency intensive applications such as when the user **104** is viewing a transcription of a voicemail left on his/her phone (e.g., FIG. 1). As discussed above, this type of application benefits from using future context to improve performance in exchange for increased processing times. Here, the model **200c** uses the first encoder **210** to generate the first higher order feature representation e.sup.s at each time step for input to the second encoder **220**, but the first decoder **204** does not decode any of the first higher order feature representations e.sup.s. The model **200c** then performs non-streaming speech recognition on all of the audio data **110** where the second encoder **220** uses the first higher order feature representation e.sup.s received from the first encoder **210** to generate the second higher order feature representation e.sup.a. The second decoder **206** then produces the final speech recognition result **120**, **120b**. Since producing streaming speech recognition in real-time has little value to the user and latency is not a factor, the model **200c** may simply operate in only the non-streaming mode to produce the final speech recognition result **120**, **120b**.

(35) Referring to FIG. 2D, the model **200** can be generalized to a three-pass model **200d** (i.e., a large model) that operates only in the non-streaming mode. This non-streaming mode may occur, for instance, in non-latency intensive applications where a user **104** expects greater accuracy in speech recognition from the model **200**. As discussed above, this type of application benefits from using an increased amount of future context at the expense of increased latency for the final speech recognition result **120**, **120b**. Here, the model **200d** uses the first encoder **210** to generate the first higher order feature representation e.sup.s at each time step for input to the second encoder **220**, however the first decoder **204** does not decode any of the first higher order feature representations e.sup.s. The model **200d** next performs non-streaming speech recognition on all of the audio data **110** where the second encoder **220** uses the first higher order feature representations e.sup.s received from the first encoder **210** to generate the second higher order feature representations e.sup.a. Unlike the model **200c**, the second decoder **206** does not decode any of the second higher order feature representations e.sup.a. Rather, the model **200d** again performs non-streaming speech recognition on all of the audio data **110** where the third encoder **230** uses the second higher order feature representation e.sup.a received from the second encoder **220** to generate the third higher order feature representation e.sup.r. The third decoder **208** then produces the final speech recognition result **120**, **120b**.

(36) FIGS. 3A and 3B show an example of a two-stage training process **300**, **300a-b** for training the model **200** to dynamically operate between streaming and/or non-streaming modes of varying model sizes. In some configurations, the training process **300** executes on the remote computing device **60** of FIGS. 1A and 1B. The training process **300** obtains a plurality of training utterances **132**, **132a-n** stored in a sample database **130** and trains the model **200** on the training utterances **132** using a two-stage process. The sample database **130** may reside on the memory hardware of the remote computing device **60**. As discussed above with respect to FIG. 2A, the first encoder **210**, the second encoder **220**, and the third encoder **230** can be trained jointly or together, simplifying the training process **300**. Training the encoders **210**, **220**, **230** jointly means that the non-streaming encoder **220** may be trained directly on the output of the streaming encoder **210** (e.g., the first higher order representation e.sup.s) instead of on input acoustic features (e.g., input acoustic frames **110**), while the non-streaming encoder **230** may be trained directly on the output of the non-streaming encoder **220** (e.g., the second higher order feature representation e.sup.a) instead of on

input acoustic features (e.g., input acoustic frames **110**). Advantageously, this ensures that the model **200** learns to operate in either streaming or non-streaming modes during inference.

(37) As shown in FIGS. **3A** and **3B**, there are three processing paths for the model **200**, one for the streaming mode of the model **200b** (shown in FIG. **2B**), one for the non-streaming mode of the model **200c** (shown in FIG. **2C**), and one for the non-streaming mode of the model **200d** (shown in FIG. **2D**). Referring to FIG. **3A**, the training process **300** employs a first-stage training process **300a**. In the first stage, the training process **300a** uses cross-entropy training by forwarding the same mini-batch of training utterances **132** through each of the cascaded encoder models **200b**, **200c**, **200d** and linearly combines the losses for each cascaded encoder model **200b**, **200c**, **200d** with weights that sum to 1. In the first stage of the training process **300a**, jointly training the cascaded encoders **210**, **220**, **230** includes minimizing the weighted sum of the loss between the input processing paths.

(38) Because there are three input processing paths within training process **300**, the model's loss includes three loss functions. Specifically, the loss for the streaming mode of the model **200b** is generally defined as a summation of the negative log probabilities corresponding to the probability distribution over possible speech recognition hypotheses given the input training utterances **132**. That is, the model loss from the first encoder **210** connection to the decoder **204** is defined as, $L_{sub.s} = -\sum_{\{x.fwdarw.e.sub.s.sub.,y\}} \log P(y|e.sup.s)$. The model loss for the non-streaming mode (e.g., model **200c**) is also generally defined as a summation of the negative log probabilities corresponding to the probability distribution over possible speech recognition hypotheses given the input training utterances **132**. Therefore, the model loss from the second encoder **220** connection to the decoder **204** is defined as $L_{sub.a} = -\sum_{\{x.fwdarw.e.sub.a.sub.,y\}} \log P(y|e.sup.a)$. The model loss for the non-streaming mode (e.g., model **200d**) is also generally defined as a summation of the negative log probabilities corresponding to the probability distribution over possible speech recognition hypotheses given the input training utterances **132**. Therefore, the model loss from the third encoder **230** connection to the third decoder **206** is defined as $L_{sub.r} = -\sum_{\{x.fwdarw.e.sub.r.sub.,y\}} \log P(y|e.sup.r)$.

In some implementations, where the model **200** only includes the encoders **210**, **220** and the decoders **204**, **206**, the total loss between the two input paths for the first decoder **204** and the second decoder **206** is computed as a weighted sum of each input path loss $L = \lambda L_{sub.s} + (1 - \lambda) L_{sub.a}$ where λ is the weighting term.

(39) In the second stage, to fine-tune the model **200** initialized during the first training stage **300a** using cross-entropy training, the training process **300b** performs discriminative training. Here, the discriminative training includes fine-tuning the model **200** using minimum word error rate (MWER) criteria. During the second stage of the training process **300b**, for each training utterance **132**, training can occur in any of the streaming or non-streaming paths/passes. In other words, the input processing path is stochastically chosen as either training the model **200b**, or the model **200c**, or the model **200d** where the training process **300b** randomly samples each model **200b**, **200c**, **200d** with a probability equal to its loss weight. Here, the loss weight for the model **200b** may be 0.8, the loss weight of the model **200c** may be 0.15, and the loss weight of the model **200d** may be 0.05. By spending the most time training the path of the model **200b**, the accuracy of the model is increased without sacrificing performance in the later paths **200c**, **200d**. Thereafter, the respective decoder **204**, **206**, **208** of the sampled path performs beam searching on the training utterances **132** to generate an n-best list of hypotheses, and the negative log-likelihood is computed for all of the hypotheses and re-normalized in the n-best space to approximate the word error loss for minimization of loss in the model **200**. Thus, for each hypothesis in the n-best list generated by the respective decoder for a corresponding training utterance, a respective number of word errors relative to a ground-truth transcription of the corresponding training utterance is identified and the training process uses the MWER criteria to minimize word error rate based on the respective number of word errors identified for each hypothesis in the n-best list for the corresponding training utterance **132**. As used herein, the n-best list includes the n highest ranked hypotheses

wherein each hypotheses in the n-best list includes a candidate transcription for the same corresponding training utterance **132**. In some implementations, n is equal to four (4) such that the respective decoder **204**, **206**, **208** of the sampled path performs beam searching on the training utterances **132** to generate the top-4 hypotheses and the negative log-likelihood is computed for all the hypotheses (greater than 4) of the hypotheses and re-normalized in the top-4 space. Moreover, by sampling the training utterances **132**, the training process only needs to calculate the loss once for each training utterance **132** at each pass/path, which greatly speeds up the second stage of the training process **300b**. In some implementations, where a longer training time is tolerated, an alternative training process is employed to train each input processing path/pass with each training utterance and compute both the loss of the model **200b** and the model **200c** for each training utterance **132**.

(40) In the example shown in FIG. 3A, the training utterances **132b**, **132c** are chosen to train each of the processing paths represented by the cascaded encoders models **200b**, **200c**, **200d** in the first stage of the training process **300a**. The cascaded encoders model **200b** receives the training utterances **132b**, **132c**, and the first encoder **210** converts the training utterances **132b**, **132c** into the first higher order feature representations (e.g., audio embeddings) as output. The first decoder **204** then receives the first higher order feature representations of training utterances **132b**, **132c** as input and generates an output which is tested for its accuracy. Similarly, training utterances **132b**, **132c** are chosen to train the second processing path represented by the cascaded encoders model **200c**. The cascaded encoders model **200c** receives the training utterances **132b**, **132c**, and the first encoder **210** converts the training utterances **132b**, **132c** into the first higher order feature representations (e.g., audio embeddings) as output. The second encoder **220** receives the first higher order feature representations of training utterances **132b**, **132c** as input and generates second higher order feature representations of the training utterances **132b**, **132c** as output. The second decoder **206** then receives the second higher order feature representations of training utterances **132b**, **132c** as input and generates an output which is tested for its accuracy. Likewise, the cascaded encoders model **200d** receives the training utterances **132b**, **132c**, and the first encoder **210** converts the training utterances **132b**, **132c** into the first higher order feature representations (e.g., audio embeddings) as output. The second encoder **220** receives the first higher order feature representations of training utterances **132b**, **132c** as input and generates second higher order feature representations of the training utterances **132b**, **132c** as output. The third encoder **230** receives the second higher order feature representations of training utterances **132b**, **132c** as input and generates third higher order feature representations of training utterances **132b**, **132c** as output. The third decoder **208** then receives the third higher order feature representations of training utterances **132b**, **132c** as input and generates an output which is tested for its accuracy.

(41) As shown in FIG. 3B, in the second stage of the training process **300b**, the training utterances **132** used by the training process **300b** are randomly selected, and each pass of the model **200** is randomly selected at a probability equal to the loss weight assigned to each path of the model **200**. Based on its loss weight, the cascaded encoders model **200b** receives the training utterances **132a**, **132b**, **132c**, and the first encoder **210** converts the training utterances **132a**, **132b**, **132c** into the first higher order feature representations (e.g., audio embeddings) as output. The first decoder **204** then receives the first higher order feature representations of training utterances **132a**, **132b**, **132c** as input and generates an output which is tested for its accuracy. Based on its loss weight, the cascaded encoders model **200c** receives the training utterances **132b**, **132d**, and the first encoder **210** converts the training utterances **132b**, **132d** into the first higher order feature representations (e.g., audio embeddings) as output. The second encoder **220** receives the first higher order feature representations of training utterances **132b**, **132d** as input and generates second higher order feature representations of the training utterances **132b**, **132d** as output. The second decoder **206** then receives the second higher order feature representations of training utterances **132b**, **132d** as input and generates an output which is tested for its accuracy. Based on its loss weight, the cascaded

encoders model **200d** receives the training utterance **132c**, and the first encoder **210** converts the training utterance **132c** into the first higher order feature representation (e.g., audio embeddings) as output. The second encoder **220** receives the first higher order feature representations of training utterance **132c** as input and generates second higher order feature representations of the training utterance **132c** as output. The third encoder **230** receives the second higher order feature representations of training utterance **132c** as input and generates third higher order feature representations of training utterance **132c** as output. The third decoder **208** then receives the third higher order feature representations of training utterance **132c** as input and generates an output which is tested for its accuracy.

(42) FIG. 4 includes a flowchart of an example arrangement of operations for a method **400** of performing streaming and non-streaming speech recognition using an ASR model **200**. At operation **402**, the method **400** includes receiving, as input to the ASR model **200**, a sequence of acoustic frames **110**. At operation **404**, the method **400** also includes generating, by a first encoder **210**, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame **110** in the sequence of acoustic frames **110**. At operation **406**, the method **400** includes receiving, as input to both a first decoder **204** and a second encoder **220**, the first higher order feature representation generated by the first encoder **210** at each of the plurality of output steps.

(43) The method **400** also includes, at operation **408**, generating, by the second encoder **220**, at each of the plurality of output steps, a second higher order feature representation for a corresponding first higher order feature representation. At operation **410**, the method **400** includes generating, by the first decoder **204**, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses. The method **400** further includes, at operation **412**, receiving, as input to a second decoder **206**, the second higher order feature representation generated by the second encoder **220** at each of the plurality of output steps. At operation **414**, the method **400** also includes generating, by the second decoder **206**, at each of the plurality of output steps, a second probability distribution over possible speech hypotheses.

(44) FIG. 5 is schematic view of an example computing device **500** that may be used to implement the systems (e.g., the audio subsystem **108**, the ASR system **109**, the user interface generator **107**, and/or the model **200**) and methods (e.g., the method **400**) described in this document. The computing device **500** is intended to represent various forms of digital computers, such as laptops, desktops, workstations, personal digital assistants, servers, blade servers, mainframes, and other appropriate computers. The components shown here, their connections and relationships, and their functions, are meant to be exemplary only, and are not meant to limit implementations of the inventions described and/or claimed in this document.

(45) The computing device **500** includes a processor **510** (e.g., data processing hardware), memory **520** (e.g., memory hardware), a storage device **530**, a high-speed interface/controller **540** connecting to the memory **520** and high-speed expansion ports **550**, and a low speed interface/controller **560** connecting to a low speed bus **570** and a storage device **530**. Each of the components **510**, **520**, **530**, **540**, **550**, and **560**, are interconnected using various busses, and may be mounted on a common motherboard or in other manners as appropriate. The processor **510** (e.g., data processing hardware **12**, **62** of FIGS. **1A-1B**) can process instructions for execution within the computing device **500**, including instructions stored in the memory **520** or on the storage device **530** to display graphical information for a graphical user interface (GUI) on an external input/output device, such as display **580** coupled to high speed interface **540**. In other implementations, multiple processors and/or multiple buses may be used, as appropriate, along with multiple memories and types of memory. Also, multiple computing devices **500** may be connected, with each device providing portions of the necessary operations (e.g., as a server bank, a group of blade servers, or a multi-processor system).

(46) The memory **520** (e.g., memory hardware **14**, **64** of FIGS. **1A-1B**) stores information non-

transitorily within the computing device **500**. The memory **520** may be a computer-readable medium, a volatile memory unit(s), or non-volatile memory unit(s). The non-transitory memory **520** may be physical devices used to store programs (e.g., sequences of instructions) or data (e.g., program state information) on a temporary or permanent basis for use by the computing device **500**. Examples of non-volatile memory include, but are not limited to, flash memory and read-only memory (ROM)/programmable read-only memory (PROM)/erasable programmable read-only memory (EPROM)/electronically erasable programmable read-only memory (EEPROM) (e.g., typically used for firmware, such as boot programs). Examples of volatile memory include, but are not limited to, random access memory (RAM), dynamic random access memory (DRAM), static random access memory (SRAM), phase change memory (PCM) as well as disks or tapes.

(47) The storage device **530** is capable of providing mass storage for the computing device **500**. In some implementations, the storage device **530** is a computer-readable medium. In various different implementations, the storage device **530** may be a floppy disk device, a hard disk device, an optical disk device, or a tape device, a flash memory or other similar solid state memory device, or an array of devices, including devices in a storage area network or other configurations. In additional implementations, a computer program product is tangibly embodied in an information carrier. The computer program product contains instructions that, when executed, perform one or more methods, such as those described above. The information carrier is a computer- or machine-readable medium, such as the memory **520**, the storage device **530**, or memory on processor **510**.

(48) The high speed controller **540** manages bandwidth-intensive operations for the computing device **500**, while the low speed controller **560** manages lower bandwidth-intensive operations. Such allocation of duties is exemplary only. In some implementations, the high-speed controller **540** is coupled to the memory **520**, the display **580** (e.g., through a graphics processor or accelerator), and to the high-speed expansion ports **550**, which may accept various expansion cards (not shown). In some implementations, the low-speed controller **560** is coupled to the storage device **530** and a low-speed expansion port **590**. The low-speed expansion port **590**, which may include various communication ports (e.g., USB, Bluetooth, Ethernet, wireless Ethernet), may be coupled to one or more input/output devices, such as a keyboard, a pointing device, a scanner, or a networking device such as a switch or router, e.g., through a network adapter.

(49) The computing device **500** may be implemented in a number of different forms, as shown in the figure. For example, it may be implemented as a standard server **500a** or multiple times in a group of such servers **500a**, as a laptop computer **500b**, or as part of a rack server system **500c**.

(50) Various implementations of the systems and techniques described herein can be realized in digital electronic and/or optical circuitry, integrated circuitry, specially designed ASICs (application specific integrated circuits), computer hardware, firmware, software, and/or combinations thereof. These various implementations can include implementation in one or more computer programs that are executable and/or interpretable on a programmable system including at least one programmable processor, which may be special or general purpose, coupled to receive data and instructions from, and to transmit data and instructions to, a storage system, at least one input device, and at least one output device.

(51) A software application (i.e., a software resource) may refer to computer software that causes a computing device to perform a task. In some examples, a software application may be referred to as an “application,” an “app,” or a “program.” Example applications include, but are not limited to, system diagnostic applications, system management applications, system maintenance applications, word processing applications, spreadsheet applications, messaging applications, media streaming applications, social networking applications, and gaming applications.

(52) These computer programs (also known as programs, software, software applications or code) include machine instructions for a programmable processor, and can be implemented in a high-level procedural and/or object-oriented programming language, and/or in assembly/machine language. As used herein, the terms “machine-readable medium” and “computer-readable medium”

refer to any computer program product, non-transitory computer readable medium, apparatus and/or device (e.g., magnetic discs, optical disks, memory, Programmable Logic Devices (PLDs)) used to provide machine instructions and/or data to a programmable processor, including a machine-readable medium that receives machine instructions as a machine-readable signal. The term “machine-readable signal” refers to any signal used to provide machine instructions and/or data to a programmable processor.

(53) The processes and logic flows described in this specification can be performed by one or more programmable processors, also referred to as data processing hardware, executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by special purpose logic circuitry, e.g., an FPGA (field programmable gate array) or an ASIC (application specific integrated circuit). Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and memory devices, including by way of example semiconductor memory devices, e.g., EPROM, EEPROM, and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and CD ROM and DVD-ROM disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(54) To provide for interaction with a user, one or more aspects of the disclosure can be implemented on a computer having a display device, e.g., a CRT (cathode ray tube), LCD (liquid crystal display) monitor, or touch screen for displaying information to the user and optionally a keyboard and a pointing device, e.g., a mouse or a trackball, by which the user can provide input to the computer. Other kinds of devices can be used to provide interaction with a user as well; for example, feedback provided to the user can be any form of sensory feedback, e.g., visual feedback, auditory feedback, or tactile feedback; and input from the user can be received in any form, including acoustic, speech, or tactile input. In addition, a computer can interact with a user by sending documents to and receiving documents from a device that is used by the user; for example, by sending web pages to a web browser on a user's client device in response to requests received from the web browser.

(55) A number of implementations have been described. Nevertheless, it will be understood that various modifications may be made without departing from the spirit and scope of the disclosure. Accordingly, other implementations are within the scope of the following claims.

Claims

1. An automated speech recognition (ASR) model comprising: a first encoder configured to: receive, as input, a sequence of acoustic frames; and generate, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames; a first decoder configured to: receive, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses; a second encoder configured to: receive, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a second higher order feature representation for a

corresponding first higher order feature frame; and a second decoder configured to: receive, as input, the second higher order feature representation generated by the second encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses, wherein the first decoder and the second decoder each comprise a respective recurrent neural network-transducer (RNN-T) architecture having a same number of parameters.

2. The ASR model of claim 1, wherein the first decoder is further configured to generate partial speech recognition results based on the first probability distribution over possible speech recognition hypotheses.

3. The ASR model of claim 1, wherein the first encoder comprises a causal encoder comprising one of: a plurality of unidirectional long short-term memory (LSTM) layers; a plurality of conformer layers; or a plurality of transformer layers.

4. The ASR model of claim 1, wherein the second encoder comprises a non-causal encoder comprising one of: a plurality of unidirectional long short-term memory (LSTM) layers; a plurality of conformer layers; or a plurality of transformer layers.

5. The ASR model of claim 1, wherein the first decoder comprises: a prediction network configured to: receive, as input, a sequence of non-blank symbols output by a final softmax layer; and generate, at each of the plurality of output steps, a dense representation; and a joint network configured to: receive, as input, the dense representation generated by the prediction network at each of the plurality of output steps and the first higher order feature representation generated by the first encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, the first probability distribution over possible speech recognition hypotheses.

6. The ASR model of claim 5, wherein the prediction network comprises: a long short-term memory (LSTM)-based prediction network; or a V2 embedding look-up table.

7. The ASR model of claim 1, wherein the second decoder comprises: a prediction network configured to: receive, as input, a sequence of non-blank symbols output by a final softmax layer; and generate, at each of the plurality of output steps, a dense representation; and a joint network configured to: receive, as input, the dense representation generated by the prediction network at each of the plurality of output steps and the second higher order feature representation generated by the second encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, the second probability distribution over possible speech recognition hypotheses.

8. The ASR model of claim 7, wherein the prediction network comprises: a long short-term memory (LSTM)-based prediction network; or a V2 embedding look-up table.

9. The ASR model of claim 1, wherein the first encoder comprises a greater number of parameters than the second encoder.

10. An automated speech recognition (ASR) model comprising: a first encoder configured to: receive, as input, a sequence of acoustic frames; and generate, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames; a first decoder configured to: receive, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses; a second encoder configured to: receive, as input, the first higher order feature representation generated by the first encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a second higher order feature representation for a corresponding first higher order feature frame; a second decoder configured to: receive, as input, the second higher order feature representation generated by the second encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses; and a third encoder configured to: receive, as input, the second higher order feature representation generated by the second encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a third

higher order feature representation for a corresponding second higher order feature representation; and a third decoder configured to: receive, as input, the third higher order feature representation generated by the third encoder at each of the plurality of output steps; and generate, at each of the plurality of output steps, a third probability distribution over possible speech recognition hypotheses.

11. A computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations comprising: receiving a sequence of acoustic frames; and generating, by a first encoder, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames; generating, by a second encoder, at each of the plurality of output steps, a second higher order feature representation for a corresponding first higher order feature representation; at each of the plurality of output steps, generating, by a first prediction network of a first decoder based on a sequence of non-blank symbols output by a final softmax layer, a dense representation; generating, by a first joint network of the first decoder, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses based on the dense representation generated by the first prediction network of the first decoder; and generating, by a second decoder, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses, wherein the first decoder and the second decoder each comprise a respective recurrent neural network-transducer (RNN-T) architecture having a same number of parameters.

12. The computer-implemented method of claim 11, wherein the operations further comprise generating partial speech recognition results based on the first probability distribution over possible speech recognition hypotheses.

13. The computer-implemented method of claim 11, wherein the first encoder comprises a causal encoder comprising one of: a plurality of unidirectional long short-term memory (LSTM) layers; a plurality of conformer layers; or a plurality of transformer layers.

14. The computer-implemented method of claim 11, wherein the second encoder comprises a non-causal encoder comprising one of: a plurality of unidirectional long short-term memory (LSTM) layers; a plurality of conformer layers; or a plurality of transformer layers.

15. The computer-implemented method of claim 11, wherein the first prediction network of the first decoder comprises: a long short-term memory (LSTM)-based prediction network; or a V2 embedding look-up table.

16. The computer-implemented method of claim 11, wherein the operations further comprise, at each of the plurality of output steps: generating, by a second prediction network of the second decoder, at each of the plurality of output steps, a dense representation; receiving, as input to a second joint network of the second decoder, the dense representation generated by the second prediction network at each of the plurality of output steps and the second higher order feature representation generated by the second encoder at each of the plurality of output steps; and generating, by the second joint network of the second decoder, at each of the plurality of output steps, the second probability distribution over possible speech recognition hypotheses.

17. The computer-implemented method of claim 16, wherein the second prediction network of the second decoder comprises: a long short-term memory (LSTM)-based prediction network; or a V2 embedding look-up table.

18. The computer-implemented method of claim 11, wherein the first encoder comprises a greater number of parameters than the second encoder.

19. A computer-implemented method that when executed on data processing hardware causes the data processing hardware to perform operations comprising: receiving a sequence of acoustic frames; generating, by a first encoder, at each of a plurality of output steps, a first higher order feature representation for a corresponding acoustic frame in the sequence of acoustic frames; generating, by a second encoder, at each of the plurality of output steps, a second higher order

feature representation for a corresponding first higher order feature representation; generating, by a first decoder, at each of the plurality of output steps, a first probability distribution over possible speech recognition hypotheses; generating, by a second decoder, at each of the plurality of output steps, a second probability distribution over possible speech recognition hypotheses; receiving, as input to a third encoder, the second higher order feature representation generated by the second encoder at each of the plurality of output steps; generating, by the third encoder, at each of the plurality of output steps, a third higher order feature representation for a corresponding second higher order feature representation; receiving, as input to a third decoder, the third higher order feature representation generated by the third encoder at each of the plurality of output steps; and generating, by the third decoder, at each of the plurality of output steps, a third probability distribution over possible speech recognition hypotheses.
